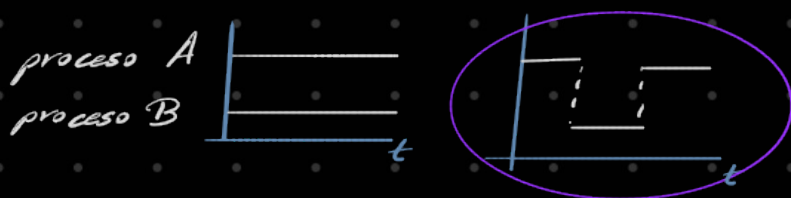


Concurrencia $\leftarrow$ Ejecución simultánea o intercalada (concurrencia simulada) de procesos.



Concurrencia simulada (pseudoparalelismo)

```
PROGRAM x;  
BEGIN  
  m; <<a>>  
  n; <<b>>
```

COBEGIN

```
  z; <<c>>  
  k; <<d>>
```

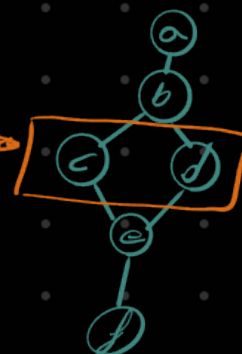
COEND;

```
  i; <<e>>
```

```
  h; <<f>>
```

END

Diagrama de precedencia



procesos concurrentes

En programación concurrente el resultado puede ser impredecible debido a la velocidad relativa de los procesos.

Aserciones

↑ Comprobaciones antes y después de una operación para controlar que se cumplan unas condiciones

$\{a * b + c = x * y \ \& \ b \neq 0\}$

WHILE (b # 0) DO BEGIN

$\{a * b + c = x * y\}$

$\{b \neq 0\}$

$b := b - 1;$

$c := c + a$

END;

$\{a * b + c = x * y \ \& \ b = 0 \rightarrow c = x * y\}$

Sistemas funcionales $\leftarrow$ Su comportamiento es independiente del tiempo.

Si los procesos operan sobre conjuntos de variables disjuntos, el sistema es funcional.

Si los procesos comparten variables, debe cumplirse cierta consistencia para garantizar un comportamiento determinista.

$\hookrightarrow$ Exclusión mutua

Algoritmo de Dekker

- Solicita el recurso

- Tratamiento de cortesía

- Por turnos

$\rightarrow$ pizarra.1 := pescando;

WHILE pizarra.2 = pescando DO

IF turno = 2

THEN BEGIN

$\rightarrow$ pizarra.1 := No pescando;

$\rightarrow$ WHILE turno = 2 DO; $\leftarrow$ espera

pizarra.1 := pescando

END;

turno := 2;

pizarra.1 := pescando;

Semáforos ← Se necesitan para dar orden a los procesos

VAR (id) : **SEMAPHORE** *tipo abstracto de datos*

→ Estructura de datos:

- contador entero positivo $[0, \infty)$
- cola de procesos esperando por ese semáforo

→ Operaciones (si s es de tipo semáforo):

- WAIT(s)
 - SIGNAL(s)
 - INIT(s, valor)
- se exigen mutuamente en el tiempo. ← Solo se pueden usar en la parte concurrente
- Solo permitida en el cuerpo principal del programa ← Parte no concurrente

INIT($s, \text{valorInicial}$) ← Inicializa el semáforo s con el valor.

WAIT(s)

contador > 0 → contador -- y el proceso continúa

contador $= 0$ → el proceso se bloquea y se encola en la cola del semáforo.

SIGNAL(s)

contador > 0 (no hay procesos esperando en la cola del semáforo s) → contador ++ y el proceso sigue.

contador $= 0$ y procesos en la cola → se despierta uno de ellos y el proceso actual sigue.

contador $= 0$ y NO hay procesos en la cola → contador ++ y el proceso actual sigue.